The US pipeline, stage by stage: command, code, files, place

Repository classification-pipeline; counts of run us-2025h1_V3 (taxonomy v0.9)

Stage and command	Code that runs	Reads → writes	Where
0 Fetch the filings `python -m airo fetch gs://airo-filings-raw --types 10-K --since 2025-01-01 --until 2025-06-30`	src/airo/fetch.py provider API, personal key in .env.local (X-API-Key; never in the cloud)	reads: the data provider's listing and filings writes: raw bucket airo-filings-raw/ markdown/<filing id>.md, manifest/<filing id>.json, companies/us.jsonl	Team member's machine → Cloud Storage
1+2 Parse and find candidate passages `python -m airo --data <dir> run <raw>/markdown --manifest <raw>/manifest [--workers N]`	src/airo/runner.py one file per worker, resumable (index/files.jsonl) → ingest.py Markdown or HTML → document record + text blocks; sections from the Form 10-K "Item" headings → retrieve.py glossary hits ± 1 block, same section; overlapping windows merge; ai-tier term → chunk, else context passage → recall.py 66 UK keyword patterns counted outside every passage glossary.py builds one regex from glossaries/core.json (502 terms, tiers ai / context, 14 groups) + glossaries/isic/<division>.json (87 files, division and class terms); container: AIRO_GLOSSARIES	reads: markdown/*.md, manifest/*.json writes: docs/<doc_id>/document.json, blocks.jsonl, passages.jsonl; index/files.jsonl V3: 7,794 filings → 28,378 chunks (5,368 filings), 64,643 context passages, 2,426 filings without a hit	Workstation or Cloud Batch container (no model)
3a Tasks and codebook `python -m airo --data <dir> tasks`	src/airo/cli.py cmd_tasks → classify.tasks() src/airo/taxonomy.py renders the codebook: every field, value, definition and criteria; TAXONOMY_VERSION = "0.9"	reads: docs/*/document.json, docs/*/passages.jsonl writes: tasks.jsonl (one task per chunk: passage, company, form, section, heading, matched terms), hints.jsonl, codebook.md	Workstation or container (no model)
3b Classification with Gemini `python -m airo.coders.gemini --tasks tasks.jsonl --codebook codebook.md --out labels_in/gemini.jsonl --model gemini-2.5-flash --thinking-budget 0 --project <project> --location us-central1 --workers 16`	src/airo/coders/gemini.py make_client() genai.Client(vertexai=True, project, location) build_batch_prompt() codebook once + 40 tasks (BATCH_SIZE) code_batch() client.models.generate_content(): temperature 0, JSON bound to response_schema = taxonomy.Labels, thinking off, 5 retries on transient errors _check() classify.validate(): field rules, quotes verbatim repair_prompt() failed label asked again with the reason; repair() or deferred and re-asked in a later batch pass	reads: tasks.jsonl, codebook.md writes: labels_in/gemini.jsonl, gemini.jsonl.run.json (tasks, failures, tokens, cost), *.deferred.jsonl V3: three passes; 49.9 M input, 7.6 M output tokens, about USD 34; 470 chunks (1.7 %) stay without label	Workstation or container → Vertex AI, us-central1 (the only model call)

3c Accept and adjudicate
`python -m airo --data <dir> accept labels_in/gemini.jsonl --coder gemini --partial`
`python -m airo --data <dir> adjudicate`
	src/airo/classify.py
	validate() pydantic Label schema (taxonomy.py), the field rules (risk_types only with role risk, …), evidence verbatim
	adjudicate() majority per field across coders; one coder on V3, so no vote; disputed fields flagged needs_review
		reads: labels_in/*.jsonl, docs/*/passages.jsonl
		writes: labels/gemini.jsonl, labels_final.jsonl, agreement.json
		V3: 27,908 labelled: 24,765 explicit, 3,143 none
			Workstation or container (no model)

4 Aggregate `python -m airo --data <dir> aggregate`	src/airo/aggregate.py one summary row per filing (ai_mention, ai_risk, ai_adoption, llm_adoption, cyber_risk, regulatory_risk, highest substantiveness, entities) and a Markdown report per filing	reads: docs/*/document.json, passages.jsonl, labels_final.jsonl writes: report_summary.jsonl, reports/<doc_id>.md, recall_check.json V3: 4,924 filings mention AI, 3,999 AI risk, 2,696 adoption	Workstation or container (no model)
5 Export the run `python -m airo --data <dir> export gs://airo-pipeline-data/us-2025h1_V3 --run-id us-2025h1_V3 --manifest <raw>/manifest`	src/airo/export.py the dataset layout kept in the bucket	reads: the working directory writes: pipeline-data bucket us-2025h1_V3/ chunks.jsonl, context_passages.jsonl, documents_manifest.json, results/classified_chunks.jsonl, filing_metrics.jsonl, codebook.md, labels_final.jsonl, agreement.json, recall_check.json, gemini.jsonl, docs.tar.gz, reports.tar.gz, README.md	→ Cloud Storage airo-pipeline-data
6 Consumers of the dataset analysis/functional_fragmentation/functional_fragmentation.py analysis/human_validation/human_validation.py dashboard/	read results/classified_chunks.jsonl, chunks.jsonl and documents_manifest.json from the bucket; model calls of the analyses go through Vertex AI in the caller's own project	writes: analysis/<name>/results/ (counts, plots, kappa); quotes and passage text stay in work/ (git-ignored) or the bucket	Workstation → Vertex AI

Deployment: Dockerfile (python 3.12, src/ + glossaries/, ENTRYPOINT python -m airo) → cloudbuild.yaml: Cloud Build trigger on every push to main runs the tests and pushes the image to Artifact Registry → a Cloud Batch job (cloud/*.job.json) mounts both buckets and runs stages 1 to 4 in order. us-2025h1_V3 was run on a workstation with the buckets as source and target; the Batch run is pending. Settings: cloud/config.env; taxonomy and codebook: src/airo/taxonomy.py; stage docs: README.md "Stages and files", docs/changes.md; dataset: docs/us-2025h1_V3-dataset.md.